

PDFBooker: Website Webscraper to Beautiful, Readable PDF Books

- **1. PROJECT SETUP & ARCHITECTURE PHASE**
 - 1.1 Set up a web application with Shadcn UI Item
 - 1.2 Create a new web application project that uses Shadcn UI components Item
 - 1.2.1 Initialize a Next.js or React project Sub-Item
 - 1.2.2 Install and configure Tailwind CSS Sub-Item
 - 1.2.3 Set up Shadcn UI Sub-Item
(<https://ui.shadcn.com/docs/installation>)
 - 1.2.4 Create the basic file structure Sub-Item
 - 1.1.4.1 pages/ or app/ directory for routes Task
 - 1.1.4.2 components/ directory for UI components Task
 - 1.1.4.3 lib/ directory for utility functions Task
 - 1.1.4.4 server/ directory for backend code Task
 - 1.1.5 Set up a basic layout with Shadcn theming Sub-Item
- **2. DESIGN THE APPLICATION ARCHITECTURE PHASE**
 - 2.1 Design the overall architecture for the web scraper to PDF application Item
 - 2.1.1 Frontend components using Shadcn UI for user input and progress display Sub-Item
 - 2.1.2. Backend server for web scraping (to avoid CORS issues)
 - 2.1.3. Content processing pipeline (scraping, cleaning, formatting)
 - 2.1.4. PDF generation module
 - 2.1.5. Data flow between components
 - 2.2 Provide a diagram or detailed explanation of how these components will interact.
- **3. FRONTEND DEVELOPMENT WITH SHADCN UI PHASE**
 - 3.1: Create the user interface with Shadcn components
 - 3.1.1 Develop a clean, responsive UI using Shadcn UI components:
 - 3.1.1.1 Use Card component for the main input form
 - 3.1.1.2 Implement Input component for the URL entry
 - 3.1.1.3 Use Slider or Select component for depth selection
 - 3.1.1.4 Add Button component for the start and download actions
 - 3.1.1.5 Implement Progress component for tracking operations
 - 3.1.1.6 Use Tabs for organizing different sections (URL input, PDF customization)
 - 3.1.1.7 Add Form components with validation
 - 3.1.1.8 Implement Accordion for advanced options
 - 3.1.1.9 Make sure the UI is responsive and follows Shadcn's design system.
 - 3.2: Implement form validation with Shadcn UI form components
 - 3.2.1 Set up form validation using Shadcn UI's form components:
 - 3.2.1.1 Implement React Hook Form integration as recommended by Shadcn UI
 - 3.2.2. Set up zod schema for validation of:
 - 3.2.2.1 - URL format (valid URL structure)

- 3.2.2.2 - Depth parameter (numeric, within reasonable range)
 - 3.2.2.3 - PDF customization options
 - 3.2.3. Add appropriate error messages using Shadcn's form error states
 - 3.2.4. Implement Toast notifications for submission feedback
 - 3.2.5 Add loading states to buttons during form submission
- **4. BACKEND DEVELOPMENT PHASE**
 - 4.1: Set up a Node.js server
 - 4.1.1 Create a Node.js server with Express that will:
 - 4.1.1.1 Handle API requests from the Shadcn UI frontend
 - 4.1.1.2 Manage the web scraping process
 - 4.1.1.3 Process and store scraped content
 - 4.1.1.4 Return the final PDF to the client
 - 4.1.2 Include appropriate error handling, CORS configuration, and logging.
 - 4.2: Implement Web Scraping Functionality Action Item
 - 4.2.1 - Develop the web crawler module that:
 - 4.2.1.1 - Takes a URL and depth as input
 - 4.2.1.2 - Recursively scrapes linked pages up to the specified depth
 - 4.2.1.3 - Handles different content types and site structures
 - 4.2.1.4 - Respects robots.txt and implements rate limiting
 - 4.2.1.5 - Stores the scraped content in an organized way
 - 4.2.1.6 - Provides status updates that can be sent to the frontend
 - 4.2.2 Use libraries like Puppeteer or Cheerio for scraping.
- **5. CONTENT PROCESSING PHASE**
 - 5.1: Create a Content Cleaner/Parser Action Item
 - 5.1.1 Build a module that processes the scraped HTML content:
 - 5.1.1.1 Extracts the main content using readability algorithms
 - 5.1.1.2 Removes ads, navigation, footers, and other non-content elements
 - 5.1.1.3 Preserves important images and formatting
 - 5.1.1.4 Normalizes and structures the content for PDF generation
 - 5.1.1.5 Maintains proper heading hierarchy and document structure
 - 5.1.2 Use libraries like Mozilla's Readability or similar tools.
 - 5.2.1 Implement Link Resolution and Content Organization Action Item
 - 5.2 Create functionality to:
 - 5.3.1 Resolve relative links within scraped content
 - 5.2.1.2 Download and include necessary images
 - 5.2.1.3 Organize content in a logical reading order

- 5.2.1.4 Generate a table of contents based on the crawled pages
 - 5.2.1.5 Create proper transitions between different pages in the final book
 - 5.2.1.6 Design a structure that will work well with PDF generation
- **6. PDF GENERATION PHASE**
 - 6.1: Develop PDF Generation Capability Action Item
 - 6.1.1 Implement a module that converts the processed content into a well-formatted PDF:
 - 6.1.1.1 Create a clean, professional book layout
 - 6.1.1.2 Include cover page, table of contents, and page numbers
 - 6.1.1.3 Support custom styling options (fonts, colors, margins)
 - 6.1.1.4 Handle images properly with appropriate sizing and resolution
 - 6.1.1.5 Generate bookmarks for different sections
 - 6.1.2 Use libraries like PDF.js, jsPDF, or html-pdf for PDF generation.
 - 6.2 Add PDF Customization Options with Shadcn UI Components
 - 6.2.1 Create features that allow users to customize their PDF output using Shadcn UI:
 - 6.2.2.1 Use Input components for book title and author information
 - 6.2.2.2 Implement ColorPicker for theme customization
 - 6.2.2.3 Add Select dropdowns for font family and size options
 - 6.2.2.4 Use Slider for margin and spacing adjustments
 - 6.2.2.5 Implement Radio or Checkbox groups for layout options
 - 6.2.2.6 Add FileInput for custom cover image upload
 - 6.2.2.7 Use Shadcn's Switch component for toggling features
 - 6.2.2 Ensure all these settings are properly applied to the PDF generation process.
- **7. INTEGRATION & USER EXPERIENCE PHASE**
 - 7.1: Connect Shadcn UI frontend with backend
 - 7.1.1 Integrate the Shadcn UI frontend with the backend API:
 - 7.1.1.1 Create API client functions to communicate with the server
 - 7.1.1.2 Use React Query or SWR for data fetching and state management
 - 7.1.1.3 Implement proper error handling with Shadcn's Alert and Toast components
 - 7.1.1.4 Show real-time progress using Shadcn's Progress component
 - 7.1.1.5 Use Skeleton components for loading states
 - 7.1.1.6 Add Dialog components for confirmation actions
 - 7.1.1.7 Implement error boundaries with fallback UI

- 7.1.2 Use Axios or fetch API for API communication.
- 7.2 Implement progress tracking with Shadcn UI
 - 7.2.1 Build a system to track and display the progress using Shadcn UI components:
 - 7.2.1.1 Use Progress component to show overall completion percentage
 - 7.1.1.2 Implement Step or custom Stepper component to show current stage
 - 7.1.1.3 Add Badge components to show status of different processes
 - 7.1.1.4 Use Accordion to display detailed progress information
 - 7.1.1.5 Implement Toast notifications for important milestones
 - 7.1.1.6 Add hover tooltips for additional information
 - 7.1.2 Ensure all progress updates are visually consistent with Shadcn's design system.
- **8. TESTING & REFINEMENT PHASE**
 - 8.1 Test with various websites
 - 8.1.1 Test the application with different types of websites:
 - 8.1.1.1 News sites with many articles
 - 8.1.1.2 Blogs with various content structures
 - 8.1.1.3 Documentation sites with complex navigation
 - 8.1.1.4 Sites with different image handling
 - 8.1.1.5 Sites with pagination and dynamic content
 - 8.1.2 Document any issues found and prioritize fixes, especially any rendering issues with Shadcn UI components.
 - 8.2 Optimize performance and reliability
 - 8.2.1 Improve the application's performance and reliability:
 - 8.2.1.1 Implement caching for scraped content
 - 8.2.1.2 Add retry mechanisms for failed requests
 - 8.2.1.3 Optimize image processing to reduce PDF size
 - 8.2.1.4 Add memory management for large crawls
 - 8.2.1.5 Implement timeout handling for slow sites
 - 8.2.2 Ensure Shadcn UI components render efficiently
- **9. DEPLOYMENT & FINAL TOUCHES PHASE**
 - 9.1 Prepare for deployment
 - 9.1.1 Get the application ready for deployment:
 - 9.1.1.1 Configure build process for the Shadcn UI frontend
 - 9.1.1.2 Set up proper environment configuration
 - 9.1.1.3 Implement security best practices
 - 9.1.1.4 Create documentation for users with Shadcn UI components
 - 9.1.1.5 Add dark mode support using Shadcn's theming
 - 9.1.1.6 Ensure accessibility compliance
 - 9.1.1.7 Test cross-browser compatibility
 - 9.1.1.8 Add final visual polish to the UI